

Desenvolvimento de Sistemas

Lista de Exercícios



Desenvolvimento de Sistemas

Lista de exercícios práticos

SENAI

2026

Sumário

Linguagem de Programação	4
Estruturas fundamentais da programação	4
Vetores	8
HashMap.....	16
POO	30
Classes, objetos e atributos.....	30
Métodos e regras de comportamento.....	31
Encapsulamento, construtores e sobrecarga	32
Herança, sobrescrita e polimorfismo	33

Linguagem de Programação

Estruturas fundamentais da programação

1. Comparação com o número 20

Desenvolva um programa que solicite ao usuário um número inteiro.

O programa deverá verificar se o número digitado é maior que 20 e apresentar uma mensagem com o resultado da comparação.

Utilize a estrutura condicional if/else.

2. Categoria de um nadador

Desenvolva um programa que solicite a idade de um nadador e informe a categoria correspondente.

Considere as seguintes regras:

- Menor que 5 anos: nenhuma categoria;
- De 5 a 7 anos: infantil;
- De 8 a 10 anos: juvenil;
- De 11 a 15 anos: adolescente;
- De 16 a 30 anos: adulto;
- Acima de 30 anos: sênior.

Utilize uma estrutura condicional encadeada com if, else if e else.

3. Área de um triângulo com validação

Desenvolva um programa que solicite a base e a altura de um triângulo e calcule sua área.

A base e a altura devem ser maiores que zero. Caso o usuário informe um valor menor ou igual a zero, o programa deverá mostrar uma mensagem de erro e solicitar novamente as duas medidas.

Utilize a estrutura do-while para realizar a validação.

A área deve ser calculada utilizando a fórmula:

$$\text{Área} = (\text{base} \times \text{altura}) \div 2$$

4. Tabuada em três etapas

Desenvolva um programa para mostrar a tabuada de um número, realizando o exercício em três etapas.

Etapa 1 – Número fixo

Defina um número diretamente no código e utilize obrigatoriamente a estrutura de repetição for para mostrar sua tabuada de 1 até 10.

Etapa 2 – Número informado pelo usuário

Modifique o programa para que o usuário possa digitar o número cuja tabuada deseja visualizar.

Continue utilizando a estrutura for para realizar as multiplicações de 1 até 10.

Etapa 3 – Validação da entrada

Utilize a estrutura do-while para garantir que o número digitado seja maior que zero.

Enquanto o usuário informar um valor menor ou igual a zero, o programa deverá solicitar uma nova entrada. Depois de receber um valor válido, deverá utilizar o for para mostrar a tabuada.

5. Dia da semana

Desenvolva um programa que solicite um número inteiro entre 1 e 7 e informe o dia da semana correspondente.

Considere:

- 1 – Domingo;
- 2 – Segunda-feira;
- 3 – Terça-feira;
- 4 – Quarta-feira;
- 5 – Quinta-feira;
- 6 – Sexta-feira;
- 7 – Sábado.

Utilize a estrutura switch.

Caso o usuário digite um valor que não esteja entre 1 e 7, o programa deverá informar que a opção é inválida.

6. Mês do ano

Desenvolva um programa que solicite um número inteiro entre 1 e 12 e informe o mês correspondente.

Considere:

- 1 – Janeiro;
- 2 – Fevereiro;

- 3 – Março;
- 4 – Abril;
- 5 – Maio;
- 6 – Junho;
- 7 – Julho;
- 8 – Agosto;
- 9 – Setembro;
- 10 – Outubro;
- 11 – Novembro;
- 12 – Dezembro.

Utilize a estrutura switch.

Caso o usuário digite um valor que não esteja entre 1 e 12, o programa deverá informar que não existe um mês correspondente.

7. Menu de operações matemáticas

Desenvolva um programa que solicite dois números e apresente o seguinte menu:

- 1 - Calcular a média dos números;
- 2 - Subtrair o menor número do maior;
- 3 - Multiplicar os números;
- 4 - Dividir o primeiro número pelo segundo.

Utilize a estrutura switch para executar a opção escolhida.

Na opção 2, utilize uma estrutura if para identificar qual dos números é o maior e realizar a subtração corretamente.

Na opção 4, verifique se o segundo número é diferente de zero antes de realizar a divisão.

Caso o usuário escolha uma opção diferente de 1, 2, 3 ou 4, o programa deverá informar que a opção é inválida.

8. Quadrado, cubo e raiz quadrada

Desenvolva um programa que leia uma quantidade não determinada de números positivos.

Para cada número informado, o programa deverá mostrar:

- O valor digitado;
- O quadrado do número;
- O cubo do número;
- A raiz quadrada do número.

Utilize a estrutura de repetição while.

O programa deverá continuar solicitando novos valores enquanto o usuário digitar números maiores que zero. A execução deverá ser finalizada quando for informado um número negativo ou igual a zero.

9. Investimentos de Carlos e João

Carlos possui R\$ 3.000,00 e investe todo esse valor em uma aplicação que rende 2% ao mês.

João possui um valor equivalente a um terço do valor inicial de Carlos e investe esse dinheiro em uma aplicação que rende 5% ao mês.

Desenvolva um programa que calcule quantos meses serão necessários para que o valor acumulado por João seja igual ou maior que o valor acumulado por Carlos.

Utilize a estrutura de repetição while.

Ao final, mostre:

- A quantidade de meses necessários;
- O valor acumulado por Carlos;
- O valor acumulado por João.

10. Menu de operações sobre o salário

Desenvolva um programa que apresente o seguinte menu:

1. Calcular o imposto sobre o salário;
2. Calcular o novo salário após um aumento;
3. Mostrar a classificação do salário;
4. Finalizar o programa.

Nas opções de 1 a 3, o programa deverá solicitar o salário bruto do funcionário e realizar a operação correspondente.

Opção 1 – Cálculo do imposto

Utilize as seguintes regras:

- Salário menor que R\$ 500,00: imposto de 5%;
- Salário de R\$ 500,00 até R\$ 850,00: imposto de 10%;
- Salário maior que R\$ 850,00: imposto de 15%.

Opção 2 – Cálculo do novo salário

Utilize as seguintes regras:

- Salário maior que R\$ 1.500,00: aumento de R\$ 250,00;
- Salário de R\$ 750,00 até R\$ 1.500,00: aumento de R\$ 50,00;
- Salário de R\$ 450,00 até R\$ 749,99: aumento de R\$ 75,00;

- Salário menor que R\$ 450,00: aumento de R\$ 100,00.

Opção 3 – Classificação do salário

Utilize as seguintes regras:

- Salário menor que R\$ 700,00: mal remunerado;
- Salário igual ou maior que R\$ 700,00: bem remunerado.

Utilize a estrutura switch para executar a opção escolhida e uma estrutura do-while para manter o menu em funcionamento até que o usuário escolha a opção 4.

Caso seja digitada uma opção diferente de 1, 2, 3 ou 4, o programa deverá informar que a opção é inválida.

Vetores

11. Lista de cidades

Nome do projeto Java: ProjetoVetoresBasicos

Pacote: exercicio11

Classe principal: ListaCidades

Método: public static void main(String[] args)

Objetivo

Criar um vetor de textos já preenchido e percorrer suas posições.

Descrição

Desenvolva um programa que:

1. Crie um vetor chamado cidades.
2. Armazene cinco nomes de cidades diretamente na declaração.
3. Exiba cada cidade utilizando um comando for.
4. Exiba também a posição de cada cidade.

```
Posição 0: Valença  
Posição 1: Barra do Pirai  
Posição 2: Vassouras  
Posição 3: Volta Redonda  
Posição 4: Resende
```


12. Cadastro de cinco números

Nome do projeto Java: ProjetoVetoresBasicos

Pacote: exercicio12

Classe principal: CadastroNumeros

Objetivo

Preencher um vetor com valores informados pelo usuário.

Descrição

Desenvolva um programa que:

1. Crie um vetor de números inteiros com cinco posições.
2. Utilize Scanner para solicitar os cinco números.
3. Armazene cada número em uma posição do vetor.
4. Depois do preenchimento, percorra novamente o vetor.
5. Exiba todos os números cadastrados.

```
Digite o número da posição 0: 15
Digite o número da posição 1: 8
Digite o número da posição 2: 21
Digite o número da posição 3: 4
Digite o número da posição 4: 10

Números cadastrados:
15
8
21
4
10
```

13. Vetores com cálculos

Exercício 13 – Soma e média das notas

Nome do projeto Java: ProjetoAnaliseNotas

Pacote: exercicio13

Classe principal: MediaNotas

Objetivo

Utilizar um vetor para armazenar notas e realizar cálculos.

Descrição

Desenvolva um programa que:

1. Crie um vetor do tipo double com quatro posições.
2. Solicite quatro notas ao usuário.
3. Armazene as notas no vetor.
4. Some todas as notas durante o preenchimento.
5. Calcule a média da turma.
6. Exiba as notas e a média final.

```
Digite a 1ª nota: 8.0
Digite a 2ª nota: 7.5
Digite a 3ª nota: 6.0
Digite a 4ª nota: 9.0

Notas cadastradas:
8.0
7.5
6.0
9.0

Média: 7.625
```

Regra

A média deve ser calculada utilizando o tamanho do vetor:

`media = soma / notas.length;`

14. Maior e menor temperatura

Nome do projeto Java: **ProjetoAnaliseTemperaturas**

Pacote: **exercicio14**

Classe principal: **AnaliseTemperaturas**

Objetivo

Encontrar o maior e o menor valor armazenado em um vetor.

Descrição

Desenvolva um programa que:

1. Crie um vetor para armazenar sete temperaturas.
2. Solicite uma temperatura para cada dia da semana.
3. Depois do preenchimento, considere inicialmente:
4. a primeira temperatura como a maior;
5. a primeira temperatura como a menor.
6. Percorra o vetor para identificar a maior e a menor temperatura.
7. Exiba todos os valores cadastrados.
8. Exiba a maior e a menor temperatura.

```
Temperaturas cadastradas:  
25.0  
27.5  
23.0  
29.0  
28.5  
22.0  
26.0  
  
Maior temperatura: 29.0  
Menor temperatura: 22.0
```

15. Vetores com condicionais

Contagem de números pares e ímpares

Nome do projeto Java: ProjetoClassificacaoNumeros

Pacote: exercicio15

Classe principal: ParesElmpares

Objetivo

Mesclar vetores, repetição e estruturas condicionais.

Descrição

Desenvolva um programa que:

1. Crie um vetor com oito números inteiros.
2. Solicite os valores ao usuário.
3. Percorra o vetor depois do preenchimento.
4. Utilize uma estrutura if para verificar se cada número é par ou ímpar.
5. Exiba a classificação de cada número.
6. Ao final, mostre:
 - quantidade de números pares;
 - quantidade de números ímpares.

```
Número 10: par  
Número 7: ímpar  
Número 4: par  
Número 13: ímpar  
  
Quantidade de pares: 2  
Quantidade de ímpares: 2
```

16. Introdução ao ArrayList

Diferentemente do vetor tradicional, o ArrayList não possui tamanho fixo. Sua quantidade de elementos pode aumentar ou diminuir durante a execução.

Cadastro dinâmico de tarefas

Nome do projeto Java: ProjetoListaTarefas

Pacote: exercicio16

Classe principal: CadastroTarefas

Objetivo

Cadastrar uma quantidade indefinida de elementos usando ArrayList.

Descrição

Desenvolva um programa que:

1. Crie um ArrayList<String> chamado tarefas.
2. Utilize um do-while para solicitar tarefas.
3. Adicione cada tarefa com o método add().
4. Pergunte se o usuário deseja cadastrar outra tarefa.
5. Continue enquanto a resposta for "s".
6. Depois do cadastro, exiba todas as tarefas usando for-each.
7. Exiba também a quantidade de tarefas cadastradas.

```
Digite uma tarefa: Corrigir atividades
Deseja cadastrar outra tarefa? s

Digite uma tarefa: Preparar aula
Deseja cadastrar outra tarefa? n

Tarefas cadastradas:
Corrigir atividades
Preparar aula
```

17. Lista de convidados

Nome do projeto Java: ProjetoListaConvidados

Pacote: exercicio17

Classe principal: ListaConvidados

Objetivo

Utilizar métodos básicos de manipulação de um ArrayList.

Descrição

O programa deverá apresentar o seguinte menu:

- 1 - Adicionar convidado
- 2 - Alterar convidado
- 3 - Remover convidado
- 4 - Procurar convidado
- 5 - Exibir convidados
- 6 - Encerrar

Regras

Opção 1

Solicitar um nome e adicionar com:

```
convidados.add(nome);
```

Opção 2

Solicitar uma posição e um novo nome. Realizar a alteração com:

```
convidados.set(posicao, novoNome);
```

Antes de alterar, verificar se a posição é válida.

Opção 3

Solicitar um nome e verificar sua existência com:

```
convidados.contains(nome);
```

Caso exista, removê-lo.

Opção 4

Solicitar um nome e informar sua posição utilizando:

```
convidados.indexOf(nome);
```

Opção 5

Exibir todos os convidados com for-each.

Opção 6

Encerrar o programa.

Os métodos `set()`, `contains()`, `remove()` e `size()` são apresentados no material como operações fundamentais de manipulação de listas.

18. Desafio integrador

Sistema de controle de notas

Nome do projeto Java: ProjetoControleNotas

Pacote: exercicio18

Classe principal: SistemaNotas

Objetivo

Integrar `ArrayList`, repetição, menu, cálculos, busca e condicionais.

Estrutura de dados

Utilize duas listas:

```
ArrayList<String> alunos = new ArrayList<>();
```

```
ArrayList<Double> notas = new ArrayList<>();
```

O nome e a nota de um aluno devem permanecer na mesma posição das duas listas.

Menu

- 1 - Cadastrar aluno
- 2 - Listar alunos
- 3 - Procurar aluno
- 4 - Alterar nota
- 5 - Remover aluno
- 6 - Exibir média da turma
- 7 - Exibir maior e menor nota
- 8 - Exibir situação dos alunos
- 9 - Encerrar

Regras do sistema

Cadastrar aluno

Solicitar:

- nome;
- nota.

Adicionar o nome na lista alunos e a nota na lista notas.

Listar alunos

Exibir:

Posição 0 - Ana - Nota: 8.5

Posição 1 - Bruno - Nota: 6.0

Procurar aluno

Solicitar o nome e utilizar indexOf().

Caso o retorno seja -1, o aluno não foi encontrado.

Alterar nota

Solicitar o nome do aluno.

Depois de encontrar sua posição, alterar a nota na mesma posição da lista notas.

Remover aluno

Remover o nome e a nota utilizando o mesmo índice.

```
alunos.remove(posicao);
```

```
notas.remove(posicao);
```

Média da turma

Somar todas as notas e dividir por:

```
notas.size()
```

Antes do cálculo, verificar se a lista está vazia com isEmpty().

Situação dos alunos

Utilizar as seguintes condições:

- nota maior ou igual a 7: aprovado;
- nota entre 5 e 6.9: recuperação;
- nota menor que 5: reprovado.

Desafio adicional

Ao encerrar, perguntar:

Deseja realmente encerrar e apagar os dados?

Caso a resposta seja positiva, utilizar:

```
alunos.clear();
```

```
notas.clear();
```

O material também apresenta métodos como `get()`, `indexOf()`, `remove()`, `isEmpty()` e `clear()` para acesso, busca e gerenciamento dos elementos.

HashMap

Atividade 1 - Cadastro de capitais

Nome do projeto	HashMapCapitais
Nome do pacote	br.com.senai.capitais
Classe Main	Main.java

Objetivo

Praticar a criação de um `HashMap`, inserção de dados e consulta por chave.

Estrutura do HashMap

```
HashMap<String, String> capitais = new HashMap<>();
```

Enunciado

Crie um programa que armazene estados brasileiros e suas respectivas capitais. A chave deverá representar o estado e o valor deverá representar a capital.

Dados sugeridos

Rio de Janeiro → Rio de Janeiro

São Paulo → São Paulo

Minas Gerais → Belo Horizonte

Bahia → Salvador

Paraná → Curitiba

Requisitos

Exiba todo o HashMap.

Solicite ao usuário o nome de um estado.

Exiba a capital correspondente.

Caso o estado não esteja cadastrado, exiba uma mensagem informando isso.

Atividade 2 - Estoque de produtos

Nome do projeto	HashMapEstoque
Nome do pacote	br.com.senai.estoque
Classe Main	Main.java

Objetivo

Utilizar um HashMap para armazenar valores numéricos e realizar alterações nos dados.

Estrutura do HashMap

```
HashMap<String, Integer> estoque = new HashMap<>();
```

Enunciado

Crie um sistema simples para controlar a quantidade de produtos disponíveis em estoque.

Dados sugeridos

Teclado → 10

Mouse → 15

Monitor → 6

Notebook → 4

Métodos que deverão ser utilizados

```
put()  
get()  
containsKey()  
remove()
```

Requisitos

Cadastre pelo menos quatro produtos.

Exiba todos os produtos e suas quantidades.

Pesquise a quantidade de determinado produto.

Altere a quantidade de um produto.

Remova um produto do estoque.

Exiba novamente o estoque atualizado.

Atividade 3 - Notas dos alunos

Nome do projeto	HashMapNotas
Nome do pacote	br.com.senai.notas
Classe Main	Main.java

Objetivo

Percorrer os elementos de um HashMap e realizar operações com seus valores.

Estrutura do HashMap

```
HashMap<String, Double> notas = new HashMap<>();
```

Enunciado

Crie um programa para armazenar o nome de alunos e suas respectivas notas. Cadastre pelo menos cinco alunos.

Exemplo de saída

```
Aluno: Carlos  
Nota: 8.5  
Situação: Aprovado
```

Requisitos

Percorra o HashMap e apresente nome, nota e situação de cada aluno.

Considere aprovado o aluno com nota maior ou igual a 7,0 e reprovado o aluno com nota menor que 7,0.

Ao final, apresente a maior nota, a menor nota e a média da turma.

Atividade 4 - Agenda telefônica

Nome do projeto	AgendaHashMap
Nome do pacote	br.com.senai.agenda
Classe Main	Main.java

Objetivo

Criar uma pequena aplicação de cadastro utilizando HashMap, Scanner e menu.

Estrutura do HashMap

```
HashMap<String, String> agenda = new HashMap<>();
```

Enunciado

Desenvolva uma agenda telefônica em que a chave seja o nome da pessoa e o valor seja seu telefone.

Menu

```
1 - Cadastrar contato  
2 - Pesquisar contato  
3 - Alterar telefone  
4 - Remover contato  
5 - Listar contatos  
0 - Encerrar
```

Requisitos

O programa deverá continuar sendo executado enquanto o usuário não escolher a opção 0.

Antes de cadastrar um contato, verifique se ele já existe utilizando `containsKey()`.

Atividade 5 - Cadastro de produtos com objetos

Nome do projeto	CadastroProdutosHashMap
Nome do pacote	br.com.senai.produtos
Classe Main	Main.java

Estrutura do HashMap

```
HashMap<Integer, Produto> produtos = new HashMap<>();
```

A chave será o código do produto e o valor será um objeto da classe Produto.

Classe Produto

Nome do arquivo: Produto.java

Atributos

```
private String nome;
private double preco;
private int quantidade;
```

Construtor

```
public Produto(String nome, double preco, int quantidade)
```

Métodos

```
getNome()
getPreco()
getQuantidade()
setNome()
setPreco()
setQuantidade()
exibirDados()
```

Enunciado e requisitos

Cadastre pelo menos quatro produtos.

Liste todos os produtos cadastrados.

Solicite um código ao usuário.

Pesquise o produto pelo código.

Exiba os dados do produto encontrado.

Caso o código não exista, informe ao usuário.

Exemplo de criação e armazenamento do objeto

```
produtos.put(1, new Produto("Teclado", 120.00, 10));
```

Atividade 6 - Sistema de alunos

Nome do projeto	SistemaAlunosHashMap
Nome do pacote	br.com.senai.alunos
Classe Main	Main.java

Estrutura do HashMap

```
HashMap<Integer, Aluno> alunos = new HashMap<>();
```

A chave representa a matrícula do aluno. O método `verificarSituacao()` deverá retornar "Aprovado" quando a nota for maior ou igual a 7 e "Reprovado" caso contrário.

Classe Aluno

Nome do arquivo: Aluno.java

Atributos

```
private String nome;  
private String curso;  
private double nota;
```

Construtor

```
public Aluno(String nome, String curso, double nota)
```

Métodos

```
getNome()  
getCurso()  
getNota()  
setNome()  
setCurso()  
setNota()  
verificarSituacao()  
exibirDados()
```

Enunciado e requisitos

No cadastro, solicite matrícula, nome, curso e nota.

Crie o objeto utilizando o construtor e armazene-o no HashMap.

Ao listar os alunos, apresente também a situação de cada um.

Exemplo de criação e armazenamento do objeto

```
Aluno aluno = new Aluno(nome, curso, nota);  
alunos.put(matricula, aluno);
```

Menu sugerido

```
1 - Cadastrar aluno  
2 - Pesquisar aluno  
3 - Listar alunos  
4 - Alterar nota  
5 - Remover aluno  
0 - Encerrar
```


Atividade 7 - Sistema de veículos

Nome do projeto	EstacionamentoHashMap
Nome do pacote	br.com.senai.estacionamento
Classe Main	Main.java

Estrutura do HashMap

```
HashMap<Integer, Veiculo> veiculos = new HashMap<>();
```

A chave será um código numérico utilizado para identificar cada veículo dentro do estacionamento.

Classe Veiculo

Nome do arquivo: Veiculo.java

Atributos

```
private String placa;  
private String modelo;  
private String proprietario;  
private int ano;
```

Construtor

```
public Veiculo(String placa, String modelo, String proprietario, int ano)
```

Métodos

```
getPlaca()  
getModelo()  
getProprietario()  
getAno()  
setPlaca()  
setModelo()  
setProprietario()  
setAno()  
exibirDados()
```

Enunciado e requisitos

Registre a entrada de veículos utilizando objetos da classe Veiculo.

Permita pesquisar e listar os veículos cadastrados.

Permita alterar o proprietário.

Na saída, remova o veículo do HashMap.

Exemplo de criação e armazenamento do objeto

```
Veiculo veiculo = new Veiculo(placa, modelo, proprietario, ano);  
veiculos.put(codigo, veiculo);
```

Menu sugerido

- 1 - Registrar entrada de veículo
- 2 - Pesquisar veículo pelo código
- 3 - Listar veículos
- 4 - Alterar proprietário
- 5 - Registrar saída do veículo
- 0 - Encerrar

Atividade 8 - Sistema de pedidos

Nome do projeto	SistemaPedidosHashMap
Nome do pacote	br.com.senai.pedidos
Classe Main	Main.java

Estrutura do HashMap

```
HashMap<Integer, Pedido> pedidos = new HashMap<>();
```

A chave será o número do pedido. O método `calcularTotal()` deverá retornar `quantidade * valorUnitario`.

Classe Pedido

Nome do arquivo: `Pedido.java`

Atributos

```
private String cliente;  
private String produto;  
private int quantidade;  
private double valorUnitario;
```

Construtor

```
public Pedido(String cliente, String produto, int quantidade, double valorUnitario)
```

Métodos

```
getCliente()  
getProduto()  
getQuantidade()  
getValorUnitario()  
setCliente()  
setProduto()  
setQuantidade()  
setValorUnitario()  
calcularTotal()  
exibirDados()
```

Enunciado e requisitos

Permita cadastrar, pesquisar e listar pedidos.

Permita alterar a quantidade e cancelar um pedido.

Para calcular o faturamento total, percorra todos os pedidos e some `pedido.calcularTotal()`.

Exemplo de criação e armazenamento do objeto

```
Pedido pedido = new Pedido(cliente, produto, quantidade, valorUnitario);  
pedidos.put(numeroPedido, pedido);
```

Menu sugerido

```
1 - Cadastrar pedido  
2 - Pesquisar pedido  
3 - Listar pedidos  
4 - Alterar quantidade  
5 - Cancelar pedido  
6 - Exibir faturamento total  
0 - Encerrar
```

Atividade 9 - Desafio: gerenciamento de funcionários

Nome do projeto	SistemaFuncionariosHashMap
Nome do pacote	br.com.senai.funcionarios
Classe Main	Main.java

Estrutura do HashMap

```
HashMap<Integer, Funcionario> funcionarios = new HashMap<>();
```

A chave será o código do funcionário. O método `aumentarSalario(double percentual)` deverá calcular e aplicar um aumento percentual sobre o salário atual.

Classe Funcionario

Nome do arquivo: `Funcionario.java`

Atributos

```
private String nome;
private String cargo;
private double salario;
```

Construtor

```
public Funcionario(String nome, String cargo, double salario)
```

Métodos

```
getNome()
getCargo()
getSalario()
setNome()
setCargo()
setSalario()
aumentarSalario(double percentual)
exibirDados()
```

Enunciado e requisitos

Permita cadastrar, pesquisar, listar e remover funcionários.

Permita alterar o cargo e aplicar aumento salarial.

Na folha salarial, apresente quantidade de funcionários, soma dos salários, maior salário e nome do funcionário que possui o maior salário.

Exemplo de criação e armazenamento do objeto

```
Funcionario funcionario = new Funcionario(nome, cargo, salario);  
funcionarios.put(codigo, funcionario);
```

Menu sugerido

```
1 - Cadastrar funcionário  
2 - Pesquisar funcionário  
3 - Listar funcionários  
4 - Alterar cargo  
5 - Aplicar aumento salarial  
6 - Remover funcionário  
7 - Exibir folha salarial  
0 - Encerrar
```

POO

Classes, objetos e atributos

1. Lâmpada de supermercado

Objetivo: Identificar atributos relevantes e criar instâncias de uma classe.

Instruções: Crie a classe Lâmpada para representar um produto vendido em um supermercado. Defina informações que permitam identificar e descrever a lâmpada e, na classe principal, crie ao menos dois objetos com valores diferentes.

Requisitos mínimos

Criar Lâmpada.java e uma classe de teste com main.

Usar tipos adequados, como String, double, int e boolean.

Exibir no console todos os dados de cada objeto.

Verificação: Os dois objetos devem manter estados independentes e apresentar dados coerentes.

2. Livro

Objetivo: Praticar abstração básica por meio da escolha de atributos.

Instruções: Crie a classe Livro com os dados essenciais de uma obra, sem considerar venda ou empréstimo. Depois, instancie dois livros e exiba suas informações.

Requisitos mínimos

Incluir pelo menos título, autor, editora, número de páginas e ano.

Manter a classe Livro separada da classe principal.

Atribuir valores aos atributos de cada objeto.

Verificação: Cada livro deve possuir seus próprios dados e ser exibido corretamente.

3. Conta corrente — estrutura inicial

Objetivo: Representar uma entidade com atributos numéricos e lógicos.

Instruções: Crie a classe ContaCorrente contendo número da conta, saldo, indicação de conta especial e limite. Instancie duas contas e apresente seus dados.

Requisitos mínimos

Usar double para valores monetários neste exercício introdutório.

Diferenciar uma conta comum de uma conta especial.

Não implementar saque e depósito ainda.

Verificação: As contas devem possuir valores distintos e ser exibidas sem misturar seus estados.

4. Contato da agenda

Objetivo: Aplicar classes e objetos em um exemplo cotidiano.

Instruções: Crie a classe Contato e represente as informações básicas de uma pessoa na agenda do celular. Cadastre três contatos e mostre os dados no console.

Requisitos mínimos

Incluir nome, telefone e e-mail.

Criar três instâncias independentes.

Organizar a exibição para facilitar a leitura.

Verificação: Todos os contatos devem ser exibidos com dados completos.

Métodos e regras de comportamento

5. Lâmpada com métodos

Objetivo: Transferir ações do objeto para métodos da própria classe.

Instruções: Amplie a classe Lampada criando métodos para ligar, desligar e mostrar o estado atual. Faça a classe principal apenas solicitar ou executar as ações de teste.

Requisitos mínimos

Usar um atributo boolean para guardar o estado.

Criar os métodos `ligar()`, `desligar()` e `mostrarEstado()`.

Testar a sequência desligada → ligada → desligada.

Verificação: A mensagem exibida deve sempre corresponder ao estado atual da lâmpada.

6. Conta corrente com operações

Objetivo: Implementar regras de negócio simples usando métodos e decisões.

Enunciado original: [abrir exercício no GitHub](#)

Instruções: Implemente métodos para depositar, sacar, consultar o saldo e verificar o uso do cheque especial. O saque só poderá ocorrer quando houver saldo ou limite suficiente.

Requisitos mínimos

O depósito deve rejeitar valores menores ou iguais a zero.

O saque deve retornar ou exibir se a operação foi realizada.

A lógica de alteração do saldo deve ficar na classe `ContaCorrente`.

Verificação: Teste depósito válido, saque válido e saque acima do valor disponível.

7. Aluno e situação nas disciplinas

Objetivo: Integrar objetos, vetores, repetição e método de decisão.

Instruções: Crie a classe `Aluno` com nome, matrícula, curso, três disciplinas e três notas. Implemente um método que informe se o aluno foi aprovado em uma disciplina, considerando nota maior ou igual a 7.

Requisitos mínimos

Usar vetores para armazenar nomes das disciplinas e notas.

Preencher os dados com Scanner na classe principal.

Percorrer os vetores e mostrar disciplina, nota e situação.

Verificação: O resultado deve ser calculado corretamente para notas abaixo, iguais e acima de 7.

Encapsulamento, construtores e sobrecarga

8. Lâmpada encapsulada

Objetivo: Reescrever uma classe conhecida aplicando encapsulamento e construtores.

Instruções: Torne privados os atributos da classe `Lampada`. Crie getters e setters apenas quando forem necessários e forneça duas formas de construir o objeto.

Requisitos mínimos

Criar um construtor sem parâmetros e outro com parâmetros.

Impedir alteração direta dos atributos pela classe principal.

Manter `ligar()`, `desligar()` e `mostrarEstado()`.

Verificação: Instancie uma lâmpada com cada construtor e confirme que ambas funcionam.

9. Conta corrente encapsulada

Objetivo: Proteger dados financeiros e garantir um estado inicial válido.

Enunciado original: [abrir exercício no GitHub](#)

Instruções: Reescreva `ContaCorrente` com atributos privados e construtores. O saldo não deve ser alterado por um setter comum: depósitos e saques devem ser os caminhos controlados para modificar o valor.

Requisitos mínimos

Criar construtor com número da conta, saldo inicial, tipo e limite.

Validar valores de depósito e saque.

Disponibilizar getter para consultar saldo e demais dados necessários.

Verificação: Tente operações válidas e inválidas e confirme que o objeto nunca entra em estado incoerente.

10. Aluno encapsulado

Objetivo: Combinar vetores, validação, encapsulamento e sobrecarga de construtores.

Enunciado original: [abrir exercício no GitHub](#)

Instruções: Reescreva a classe `Aluno` com atributos privados. Inicialize o aluno por construtor e controle o cadastro de disciplinas e notas por métodos.

Requisitos mínimos

Criar dois construtores com assinaturas diferentes.

Aceitar notas somente entre 0 e 10.

Manter o método de verificação de aprovação.

Usar getters apenas para consultas necessárias.

Verificação: Teste uma nota inválida e confirme que ela não é aceita; depois exiba as situações válidas.

Herança, sobrescrita e polimorfismo

11. Livro de livraria

Objetivo: Especializar a modelagem de um livro para o contexto de venda.

Instruções: Crie a classe LivroDeLivraria. Reaproveite os atributos essenciais do exercício Livro e acrescente apenas informações necessárias para a comercialização.

Requisitos mínimos

Adicionar preço e disponibilidade em estoque.

Criar pelo menos dois objetos.

Exibir os dados bibliográficos e comerciais.

Verificação: A classe deve representar claramente um livro à venda.

12. Contas bancárias especializadas

Objetivo: Criar uma hierarquia de classes e sobrescrever comportamentos.

Instruções: Implemente ContaBancaria como superclasse e ContaPoupanca e ContaEspecial como subclasses. Reutilize os dados comuns e especialize apenas o que muda em cada tipo.

Requisitos mínimos

ContaBancaria: nome do cliente, número, saldo, sacar() e depositar().

ContaPoupanca: dia de rendimento e calcularNovoSaldo().

ContaEspecial: limite e sobrescrita de sacar().

Usar protected apenas quando necessário; prefira private com métodos de acesso.

Verificação: Crie uma instância de cada subclasse e teste saque, depósito e rendimento.

13. Imposto de renda por tipo de contribuinte

Objetivo: Aplicar herança e polimorfismo em um cálculo com regras diferentes.

Instruções: Crie a superclasse Contribuinte e as subclasses PessoaFisica e PessoaJuridica. Cada subclasse deve calcular o imposto conforme sua própria regra, sobrescrevendo um método comum.

Requisitos mínimos

Definir calcularImposto() na estrutura comum e sobrescrevê-lo nas subclasses.

Pessoa jurídica: 10% da renda bruta.

Pessoa física: aplicar as faixas e parcelas do enunciado.

Armazenar os seis contribuintes em um vetor ou ArrayList de Contribuinte.

Verificação: Percorra a coleção chamando o mesmo método para todos e confira ao menos um caso de cada faixa.

Observação didática: A coleção do tipo Contribuinte evidencia o polimorfismo: a versão executada depende do objeto real.

14. Hierarquia de animais

Objetivo: Consolidar abstração, herança, especialização e exibição polimórfica.

Enunciado original: [abrir exercício no GitHub](#)

Instruções: Crie Animal como superclasse e Peixe e Mamifero como subclasses. Represente camelo, tubarão e urso-do-canadá, reutilizando os atributos comuns e incluindo as características específicas.

Requisitos mínimos

Animal: nome, comprimento, patas, cor, ambiente e velocidade.

Peixe: característica de barbatanas e cauda, com padrões do enunciado.

Mamifero: ambiente terrestre; represente o urso com seu alimento preferido.

Criar um método mostrarDados() e sobrescrevê-lo quando houver informação específica.

Verificação: Armazene os animais em uma coleção de Animal, percorra-a e produza uma saída semelhante à tabela do enunciado.

